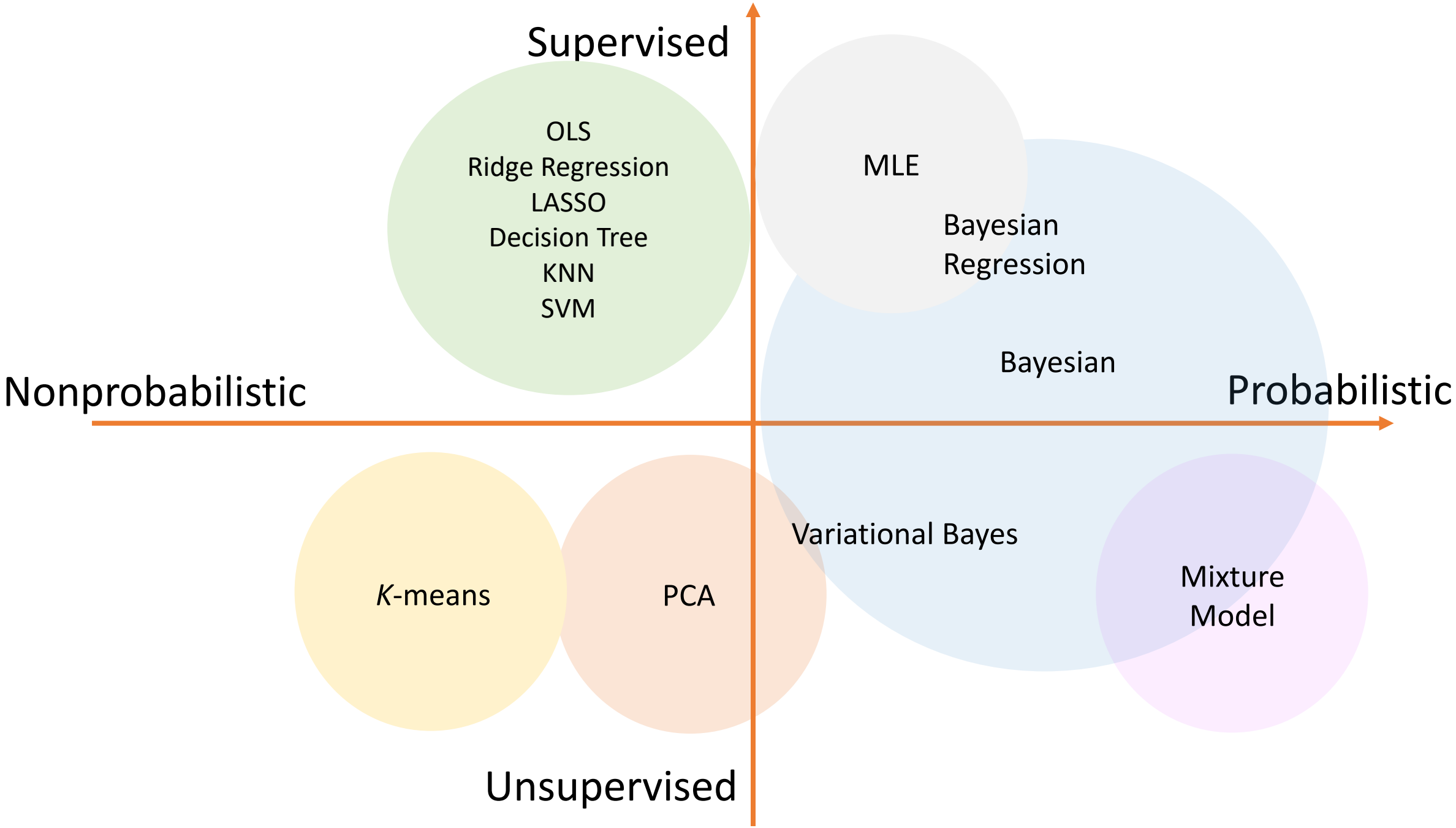
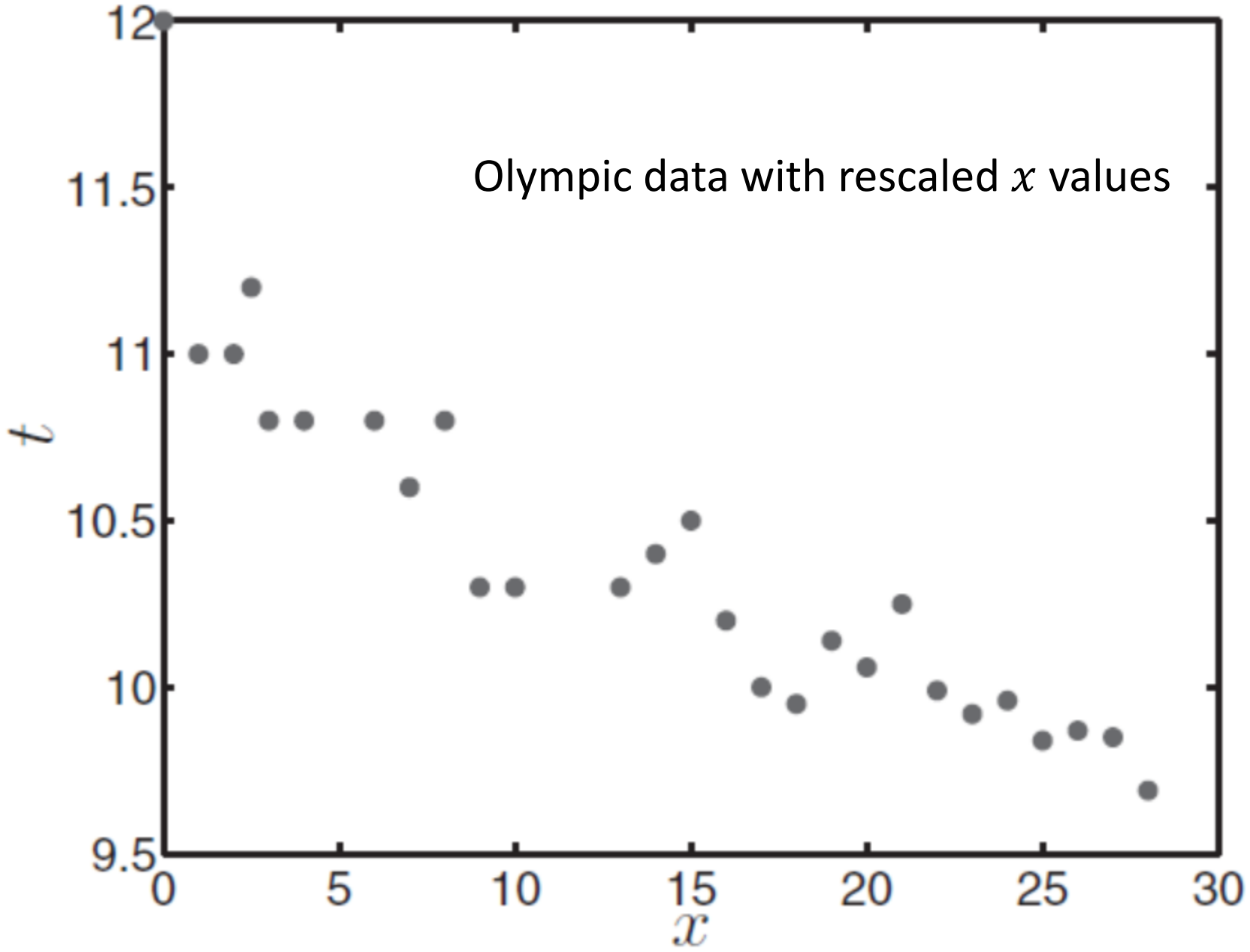

Supervised
OLS
Ridge Regression
LASSO
Decision Tree
KNN
SVM
MLE
Bayesian
Regression
Bayesian
Nonprobabilistic
Probabilistic
Variational Bayes
K-means
PCA
Mixture
Model
Unsupervised


Olympic data with rescaled x values
12
11.5
11
10.5
10
9.5
t
0
5
10
15
20
25
30
x

Gaussian Prior

- We illustrate the prior and posterior with a first-order polynomial. So the prior $p(\mathbf{w}|\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0) = N(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$ where $\boldsymbol{\mu}_0 = [0,0]^{\mathrm{T}}$ and

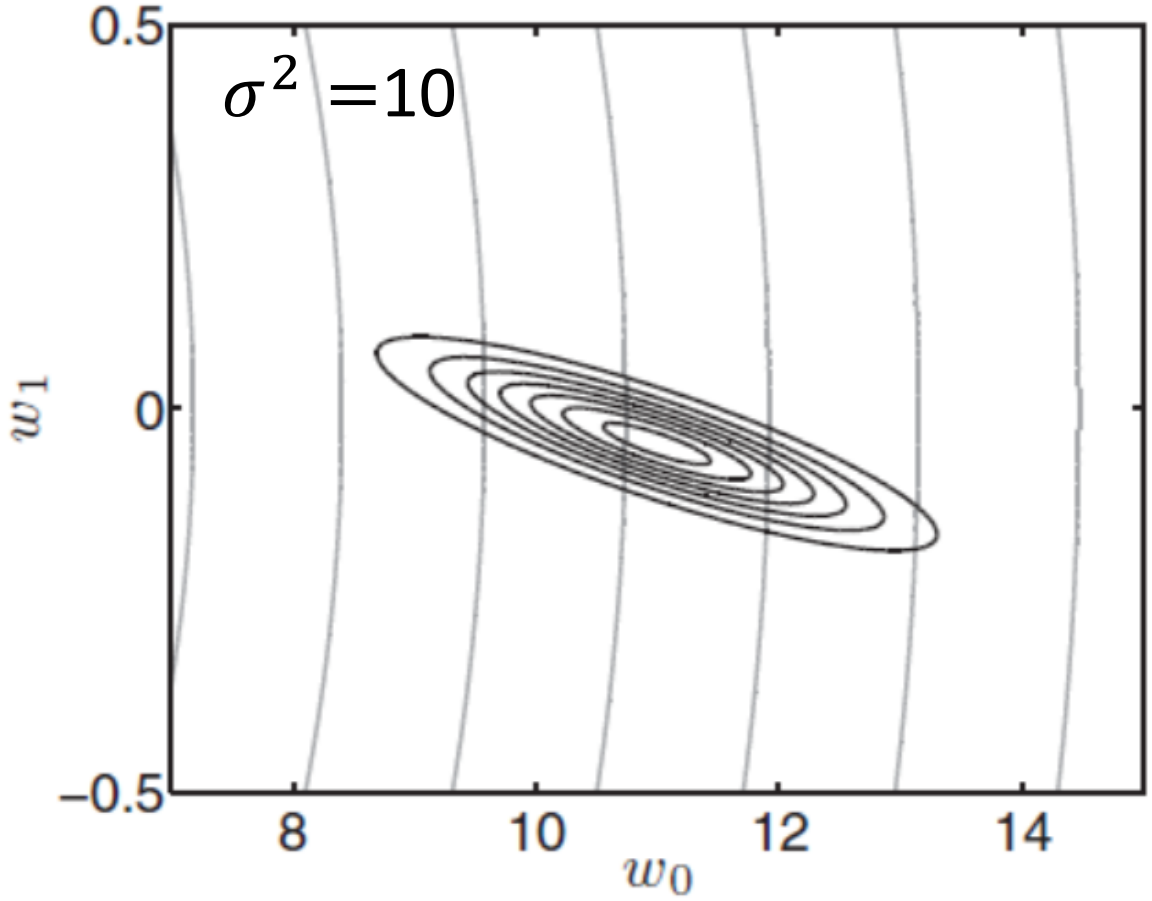
$$\boldsymbol{\Sigma}_0 = \begin{bmatrix} 100 & 0 \\ 0 & 5 \end{bmatrix}$$

- We assume the two parameters are independent. Since the MLE of w_0 is much larger than w_1, the larger value for variance is assigned to w_0.

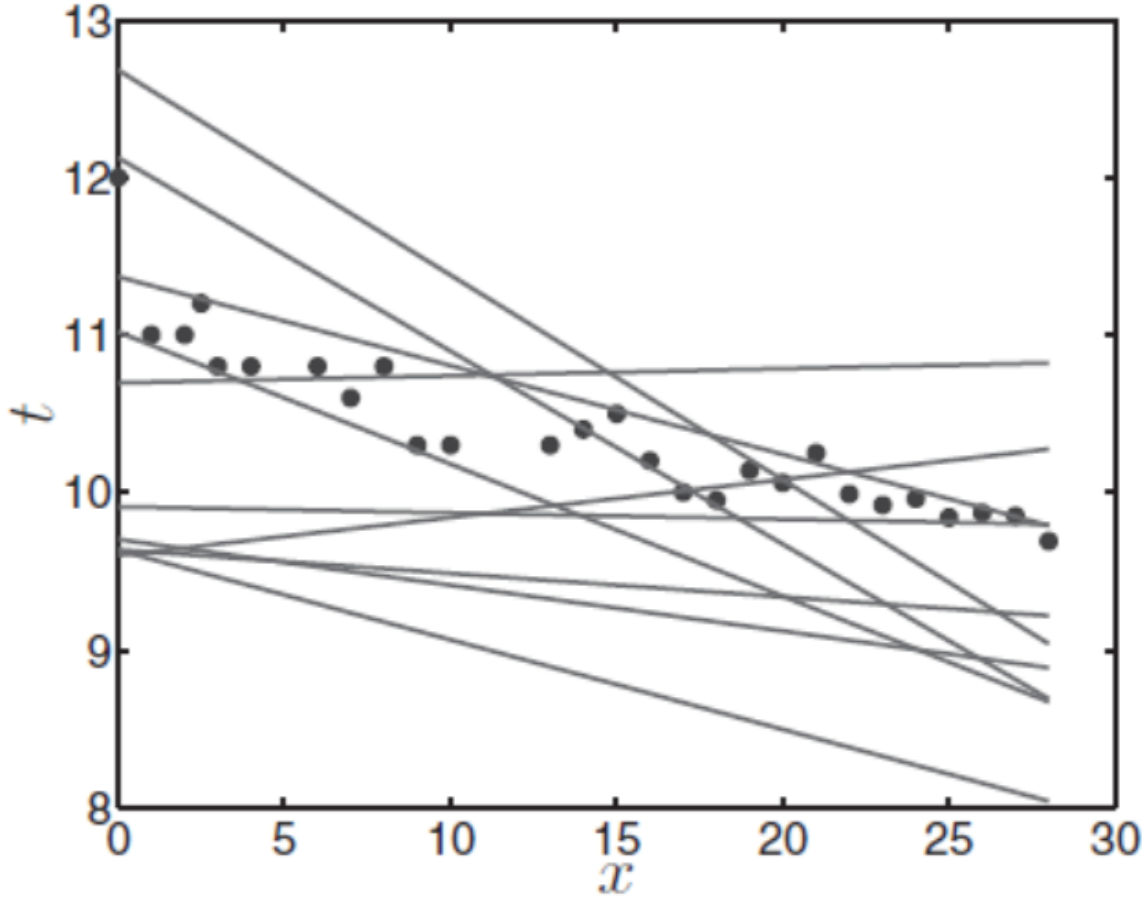
- Whether w_0 and w_1 are independent in the posterior?

Final Posterior

- After all 27 data points are included, we can see functions drawn from the final posterior.
- The functions are really now beginning to follow the trend in our data.
- There is still a lot of variability though, due to the relatively high value of $\sigma^2 = 10$ (to help visualization).



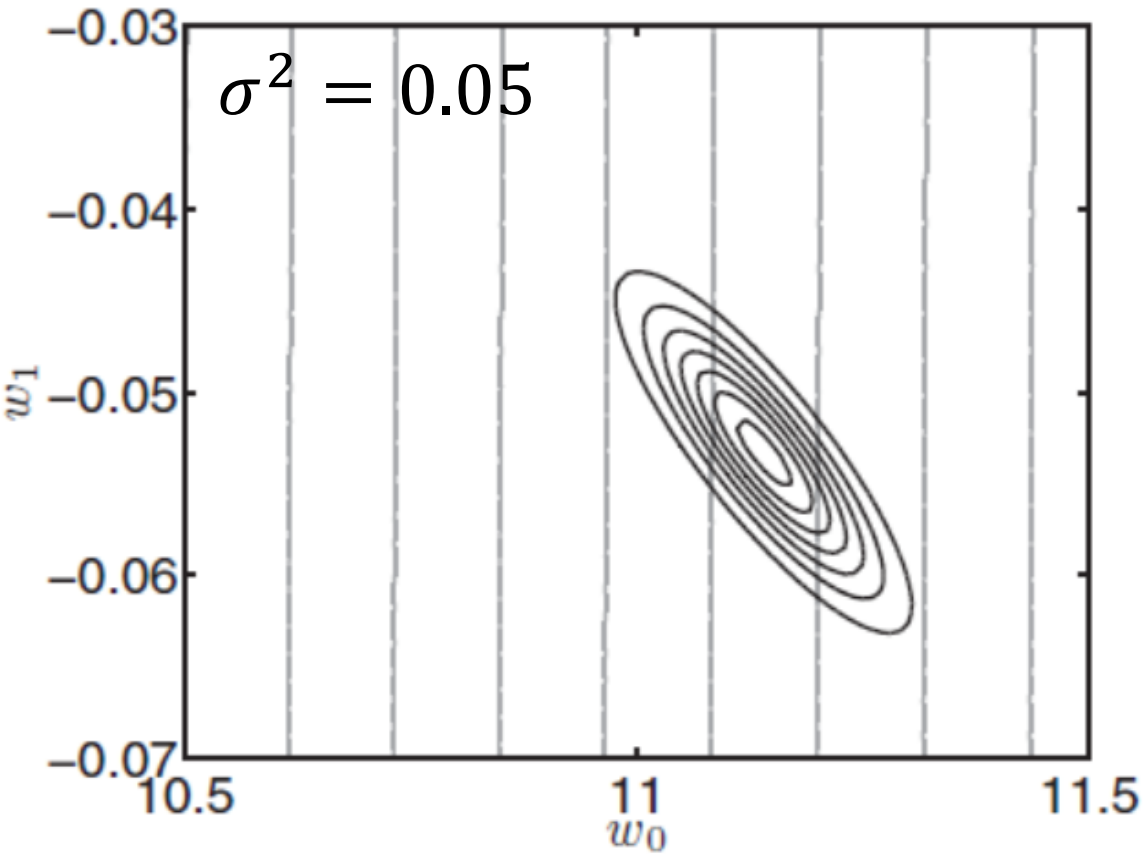
(a) Posterior density (dark contours) after all datapoints have been observed. The lighter contours show the prior density. (Note that we have zoomed in.)



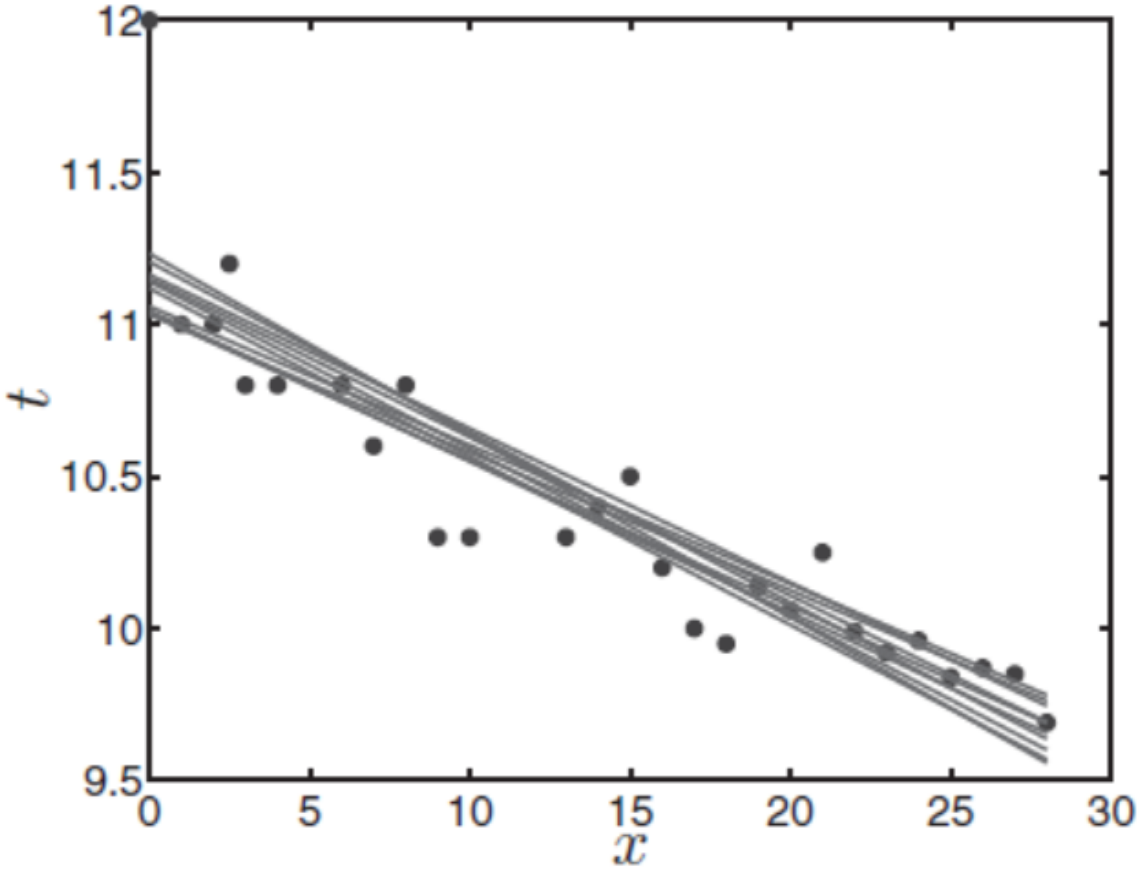
(b) Functions created from parameters drawn from the posterior after observing all data points.

Final Posterior

- For making predictions, we will use a more realistic value $\sigma^2 = 0.05$.
- The posterior is now far more condensed—very little variability remains in $\mathbf{w}$.
- This can be presented by the homogeneity of the set of functions created from parameters drawn from the final posterior.



(a) Posterior density (dark contours) after all data points have been observed. The lighter contours show the prior density. (Note that we have zoomed in.)



(b) Functions created from parameters drawn from the posterior after observing all data points.

Making Predictions

- Given $\mathbf{x}_{\text{new}} = [1, 29]^{\text{T}}$, we are interested in the predictive density $p(t_{\text{new}}|\mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2)$ which is not conditioned on $\mathbf{w}$.

- Analogous to Eq. 3.9, predictions are yielded by integrating out $\mathbf{w}$ as an expectation w.r.t. the posterior:

$$p(t_{\text{new}}|\mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) = \mathbf{E}_{p(\mathbf{w}|\mathbf{t},\mathbf{x},\sigma^2)}\{p(t_{\text{new}}|\mathbf{x}_{\text{new}}, \mathbf{w}, \sigma^2)\}$$

$$= \int p(t_{\text{new}}|\mathbf{x}_{\text{new}}, \mathbf{w}, \sigma^2)p(\mathbf{w}|\mathbf{X}, \mathbf{t}, \sigma^2)\, d\mathbf{w}$$

posterior

Bayesian Predictive Density

- Our model is defined as the product of $\mathbf{x}_{\text{new}}$ and $\mathbf{w}$ with some Gaussian noise:

$$p(t_{\text{new}}|\mathbf{x}_{\text{new}}, \mathbf{w}, \sigma^2) = N(\mathbf{w}^{\text{T}}\mathbf{x}_{\text{new}}, \sigma^2)$$

- Since this likelihood and the posterior are both Gaussian, the result of the expectation is another Gaussian.

- Given the posterior $p(\mathbf{w}|\boldsymbol{\mu}_{\mathbf{W}}, \boldsymbol{\Sigma}_{\mathbf{W}}) = N(\boldsymbol{\mu}_{\mathbf{W}}, \boldsymbol{\Sigma}_{\mathbf{W}})$, the predictive density is given by

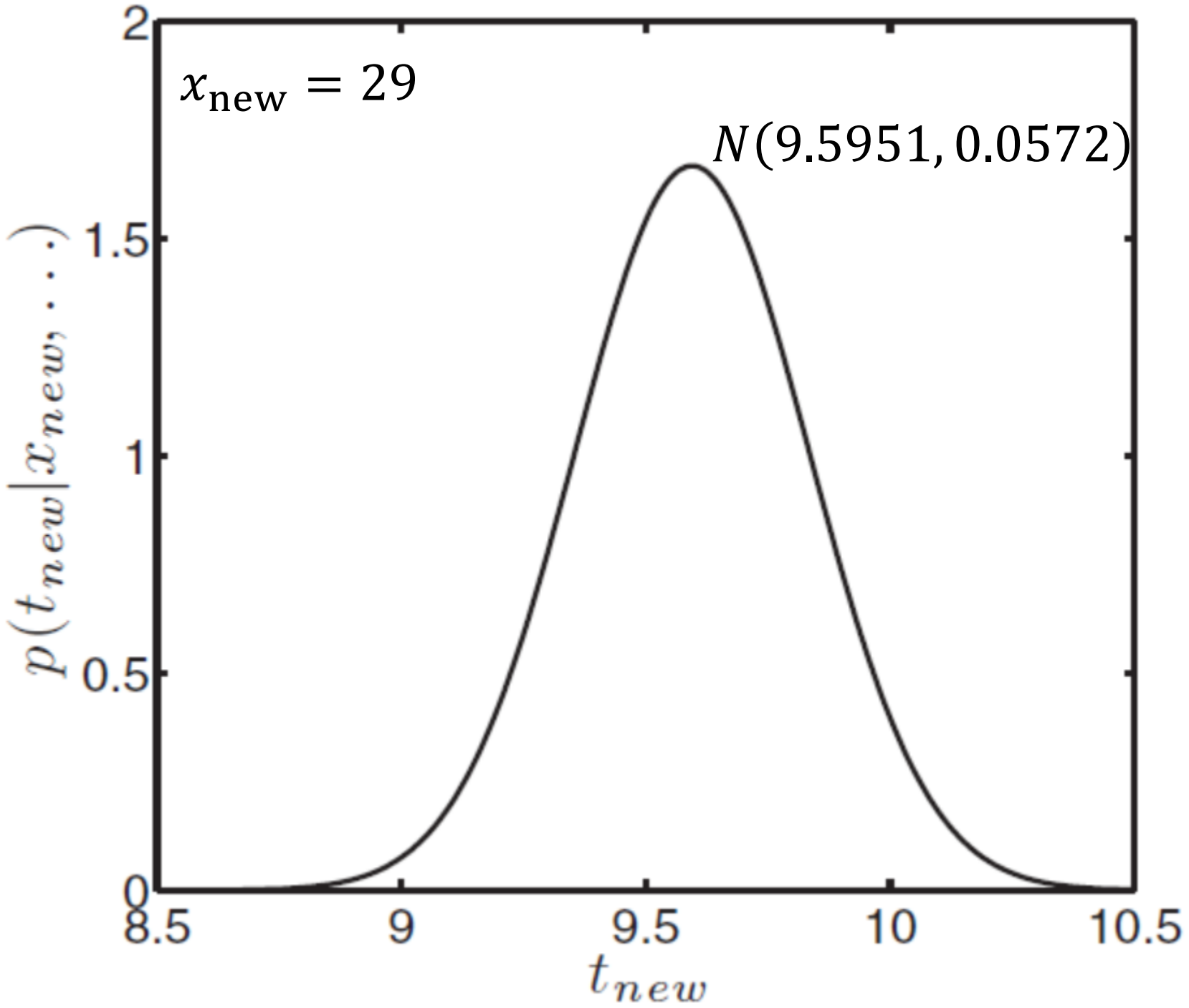
$$p(t_{\text{new}}|\mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) = N\left(\mathbf{w}^{\text{T}}\boldsymbol{\mu}_{\mathbf{W}}, \sigma^2 + \mathbf{x}_{\text{new}}^{\text{T}}\boldsymbol{\Sigma}_{\mathbf{w}}\mathbf{x}_{\text{new}}\right)$$

Bayesian Predictive Density

- For $\sigma^2 = 0.05$, our prediction for $\mathbf{x}_{\text{new}} = [1, 29]^{\text{T}}$ is
$$p(t_{\text{new}}|\mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) = N(9.5951, 0.0572).$$

- Though both are normal distributed, the MLE chooses *only* the one corresponding to the highest likelihood.

- Bayesian predictive density, by contrast, averages over all models that are consistent with our data and prior (and thus posterior).
It takes into account all uncertainty that remains in $\mathbf{w}$ *coming from a prior and the data*.


$x_{\text{new}} = 29$
$N(9.5951, 0.0572)$
$p(t_{new}|x_{new}, \ldots)$
2
1.5
1
0.5
0
8.5
9
9.5
10
10.5
t_{new}

A First Course in Machine Learning Chapter 4

Chao-En Yu

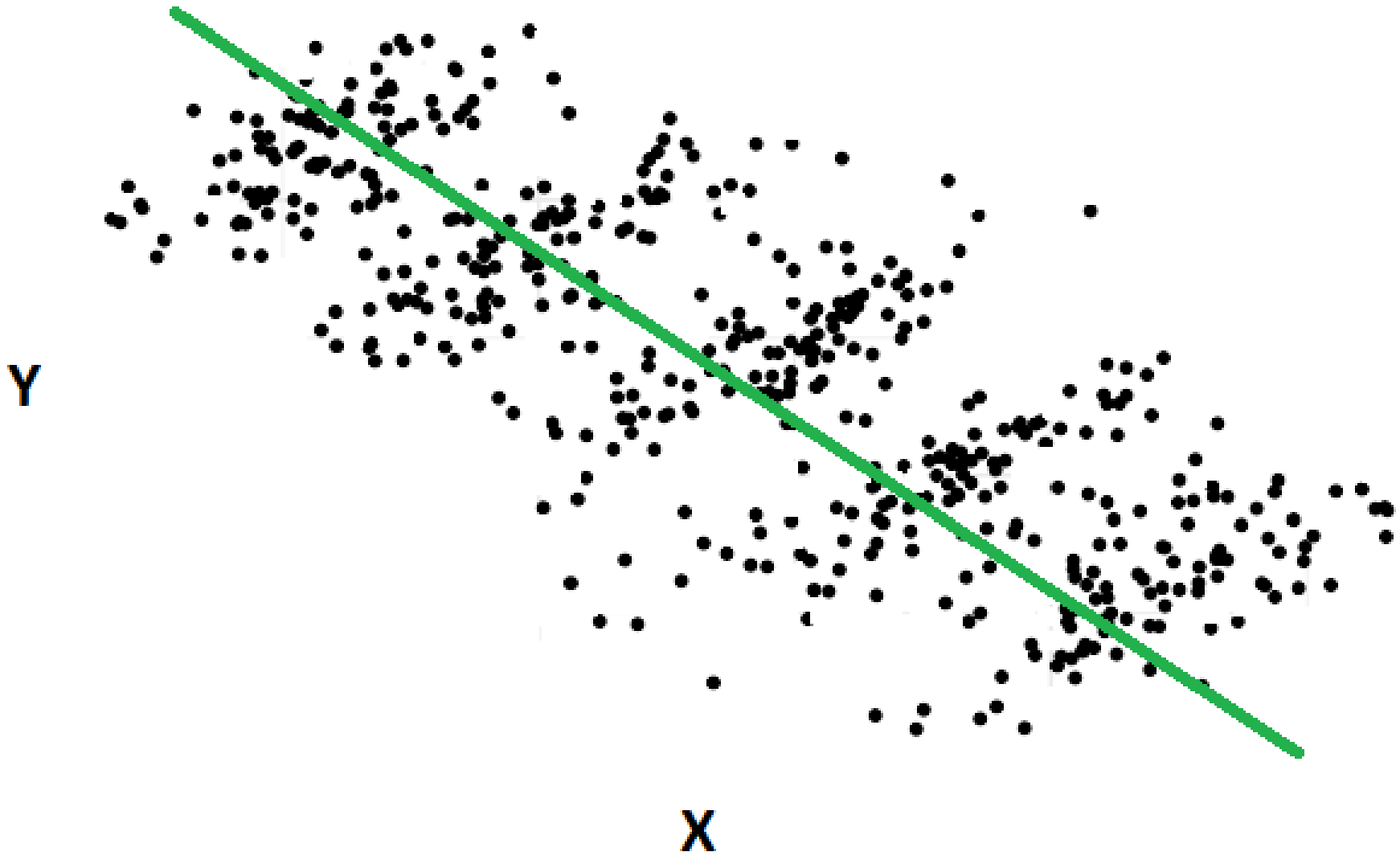
National Tsing Hua University

With No Conjugate

- Within the Bayesian framework, each parameter is described by a distribution rather than an individual value.
- Uncertainty in our parameter estimates is naturally channeled into any predictions we make.
- Examples with a justified conjugate prior and likelihood combination are rare. In this case, we *cannot* compute the posterior and must resort to approximations.

Simpson's Paradox

授課語言	統計量	2000入學生	2011入學生
中文授課	學生人數	2000	2000
	被當人數	4	2
	比率	0.002	0.001
英語授課	學生人數	600	2500
	被當人數	9	25
	比率	0.015	0.01
全校	學生人數	2600	4500
	被當人數	13	27
	比率	0.005	0.006

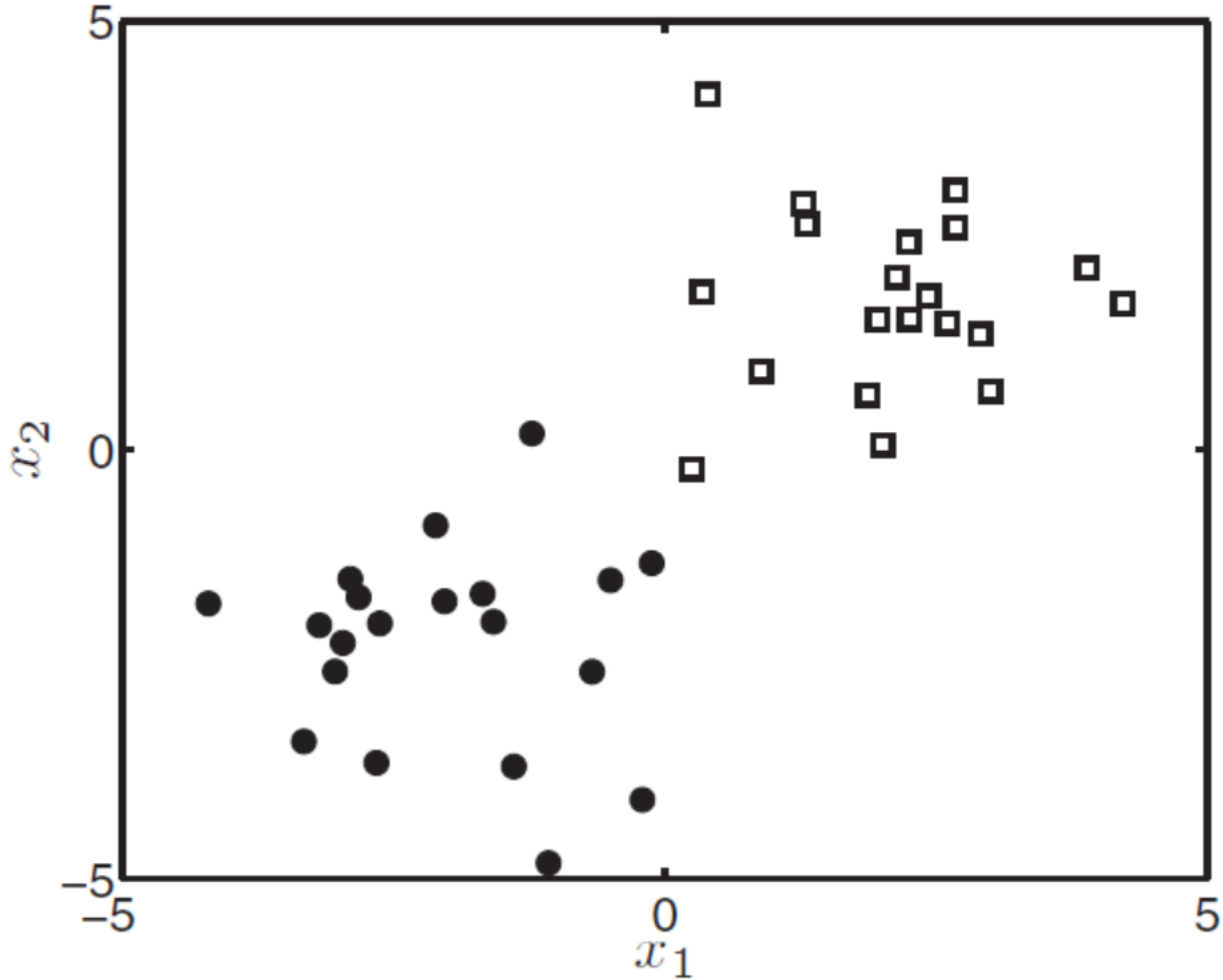

Y
X

Two-Class Classification

- Each object is described by two attributes, x_1 and x_2, and has a binary response, $t \in \{0,1\}$. If $t = 0$, the point is plotted as a circle; if $t = 1$, as a square.

- We want to be able to classify objects into one of a set of classes (in this case there are two classes). This task is known as *classification*.

- The three techniques that we will look at are a point estimate (MAP), an approximate density, and Metropolis-Hastings sampling.


5
0
−5
−5
0
5
x_2
x_1

No Intercept

- We will work with the following vector and matrix representations of our data:

$$\mathbf{x}_n = \begin{bmatrix} x_{n1} \\ x_{n2} \end{bmatrix}, \ \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \ \mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}$$

- According to Bayes' rule, the posterior is given by

$$p(\mathbf{w}|\mathbf{t}, \mathbf{X}) = \frac{p(\mathbf{t}|\mathbf{X}, \mathbf{w})p(\mathbf{w})}{p(\mathbf{t}|\mathbf{X})}$$

Sigmoid-Binomial Likelihood

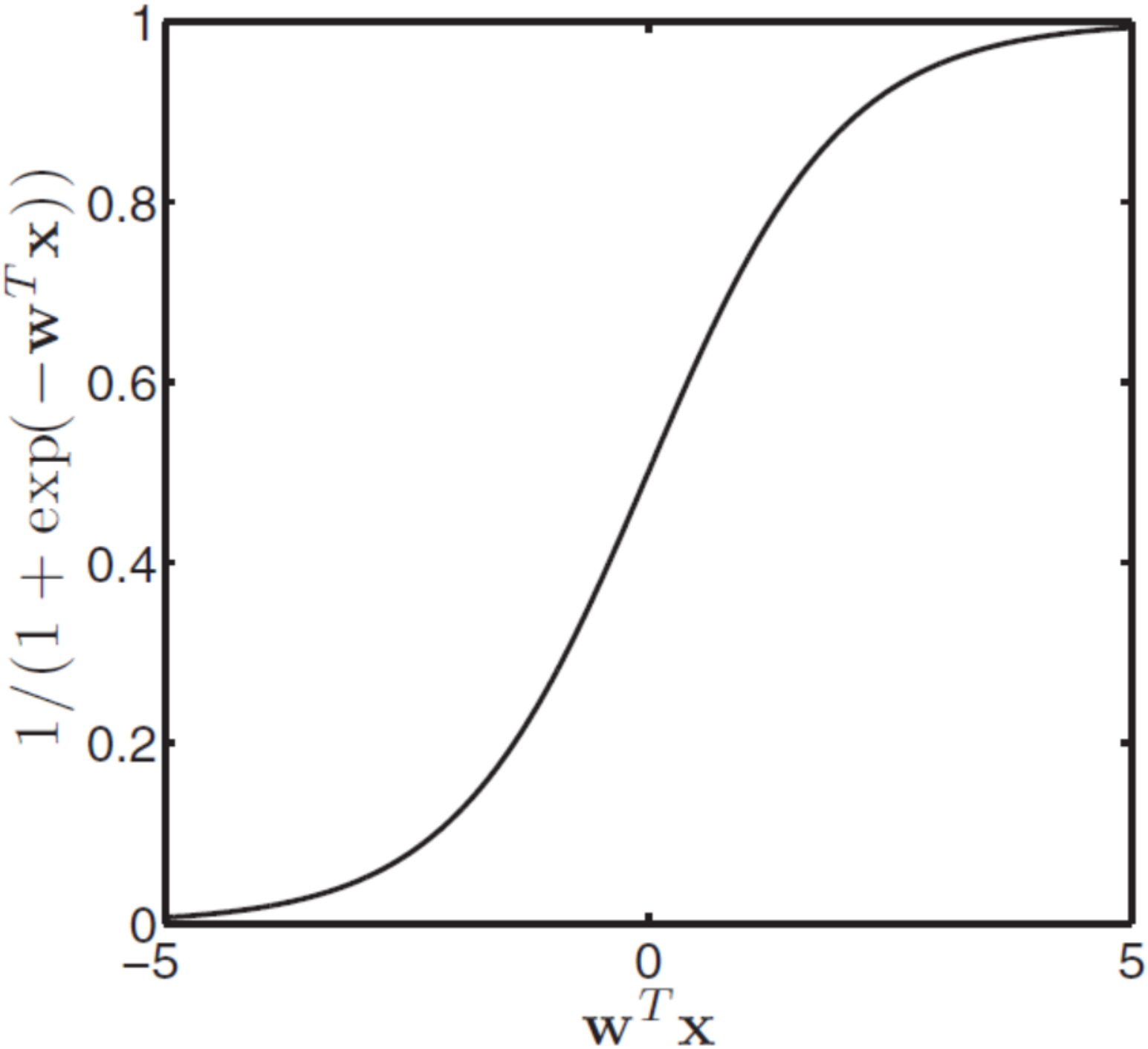
- To squash the output of $f(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)$ to be a probability, we use the sigmoid function:

$$P(T_n = 1|\mathbf{x}_n, \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)}$$

$$P(T_n = 0|\mathbf{x}_n, \mathbf{w}) = \frac{\exp(-\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)}{1 + \exp(-\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)}$$

- Combining the above equations to have

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}) = \prod_{\boldsymbol{n=1}}^{\boldsymbol{N}} \left(\frac{1}{1 + \exp(-\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)}\right)^{t_n} \left(\frac{\exp(-\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)}{1 + \exp(-\mathbf{w}^{\mathrm{T}}\mathbf{x}_n)}\right)^{1-t_n}$$


1
0.8
0.6
0.4
0.2
0
−5
0
5
$1/(1+\exp(-\mathbf{w}^T\mathbf{x}))$
$\mathbf{w}^T\mathbf{x}$

Bayesian Prediction

- By combining the above likelihood and a Gaussian prior $p(\mathbf{w}|\sigma^2) = N(\mathbf{0}, \sigma^2\mathbf{I})$, we can work out the posterior density $p(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2)$.

- Once we have the posterior, we can predict the class of new objects by taking an expectation with respect to the sigmoid function:

$$P(t_{\text{new}} = 1|\mathbf{x}_{\text{new}}, \mathbf{w}, \mathbf{t}) = \mathbf{E}_{\mathbf{w}}\left[\frac{1}{1 + \exp(-\mathbf{w}^{\mathrm{T}}\mathbf{x}_{\text{new}})}\right]$$

- Question: Why do we need to take an expectation?

Difficulty

- The bad new is that we cannot analytically perform the integration required to compute the marginal likelihood.

The numerator,
or the unnormalized posterior

- We have $g(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2) = p(\mathbf{t}|\mathbf{w}, \mathbf{X})p(\mathbf{w}|\sigma^2)$ which we know is proportional to the posterior $p(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2) = Z * g(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2)$, where

$$Z^{-1} = p(\mathbf{t}|\mathbf{X}, \sigma^2) = \int p(\mathbf{t}|\mathbf{w}, \mathbf{X})p(\mathbf{w}|\sigma^2)\, d\mathbf{w}$$

- Note that Z^{-1} is not a function of $\mathbf{w}$.

Three Options

1. MAP: Just maximizing $g(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2)$ with respect to $\mathbf{w}$. Not very Bayesian: predictions are made based on a single value of $\mathbf{w}$, rather than on a density.

2. Laplace: Approximate $p(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2)$ with another density which is easy to compute analytically.

Similar to variational Bayes in Ch.7

3. Metropolis-Hastings: Sampling directly from the $g(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2)$ is the same as sampling from the posterior $p(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2)$.

Maximum a Posteriori (MAP)

- The value of $\mathbf{w}$ that maximizes $g(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2)$ will also correspond to the value at the maximum of the posterior.

- It is easiest to find $\mathbf{w}$ that maximizes $\log g(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2)$ rather than $g(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2)$:

$$\log g(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2) = \log p(\mathbf{t}|\mathbf{X}, \mathbf{w}) + \log p(\mathbf{w}|\sigma^2)$$

- Unlike MLE, we cannot obtain an exact expression for $\mathbf{w}$ by differentiating this expression and equating it to zero. (or try sp.nsolve())

Newton-Raphson Method

- Instead, the authors start with a guess for $\mathbf{w}$ and then keep updating it with the Newton-Raphson method:

$$\mathbf{w}' = \mathbf{w} - \left(\frac{\partial^2 \log g(\mathbf{w}; \mathbf{X}, \mathbf{t})}{\partial \mathbf{w} \partial \mathbf{w}^{\mathrm{T}}} \right)^{-1} \frac{\partial \log g(\mathbf{w}; \mathbf{X}, \mathbf{t})}{\partial \mathbf{w}}$$

- Letting $P_n = P(T_n = 1 | \mathbf{w}, \mathbf{x}_n) = 1/(1 + \exp(-\mathbf{w}^{\mathrm{T}} \mathbf{x}_n))$, we have

$$\log g(\mathbf{w}; \mathbf{X}, \mathbf{t})$$

$$= \log p(\mathbf{w} | \sigma^2) + \sum_{n=1}^{N} \log P_n^{t_n} + \log(1 - P_n)^{(1-t_n)}$$

Bernoulli likelihood

What is JAX

- JAX is a Python library designed for high-performance machine learning research and numerical computing.
- JAX is developed by Google Research, specifically the Google Brain team.
- JAX includes Autograd, Vector Mapping (vmap), Just In Time compilation (JIT), all compiled with Accelerated Linear Algebra (XLA) and Tensor processing units (TPUs).

Derivatives

- Grad creates a function that evaluates the gradient of a given function:

```
import jax
import numpy as np
def f(x): return 2*x[0]**3
gradf = jax.grad(f)
gradf(np.array([2.0]))
```

- If we called grad(grad(f)), this would return the second derivative.

Newton's Method

- Newton's method is used for optimization given that we search for $f'(x) = 0$:

$$x_{n+1} = x_n - \frac{f'(x)}{f''(x)}$$

- To solve the minimum of $f(x) = (x-2)^2$:

```
def f(x): return (x-2)**2
gradf = jax.grad(f)
gradf2 = jax.grad(jax.grad(f))
def newton(x): return x-gradf(x)/gradf2(x)
newton(0.0)
```

Jacobian

- A Jacobian is used for taking the automatic derivative of a function with a vector input:

$$\nabla f(X) = \left[\frac{\partial f}{\partial x_0}, \frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n}\right]$$

- We can compute the Hessian matrix of a function by computing the Jacobian twice:

```
def hessian(f): return jax.jacfwd(jax.jacrev(f))
Hes = hessian(f)
Hes(np.array([1.0, 2.0]))
```

Hessian Matrix

- We can compute the Hessian matrix of a function by computing the Jacobian twice: `jacfwd(jacrev(f))`:

```
def hessian(f): return jax.jacfwd(jax.jacrev(f))
Hes = hessian(f)
Hes(np.array([1.0, 2.0]))
```

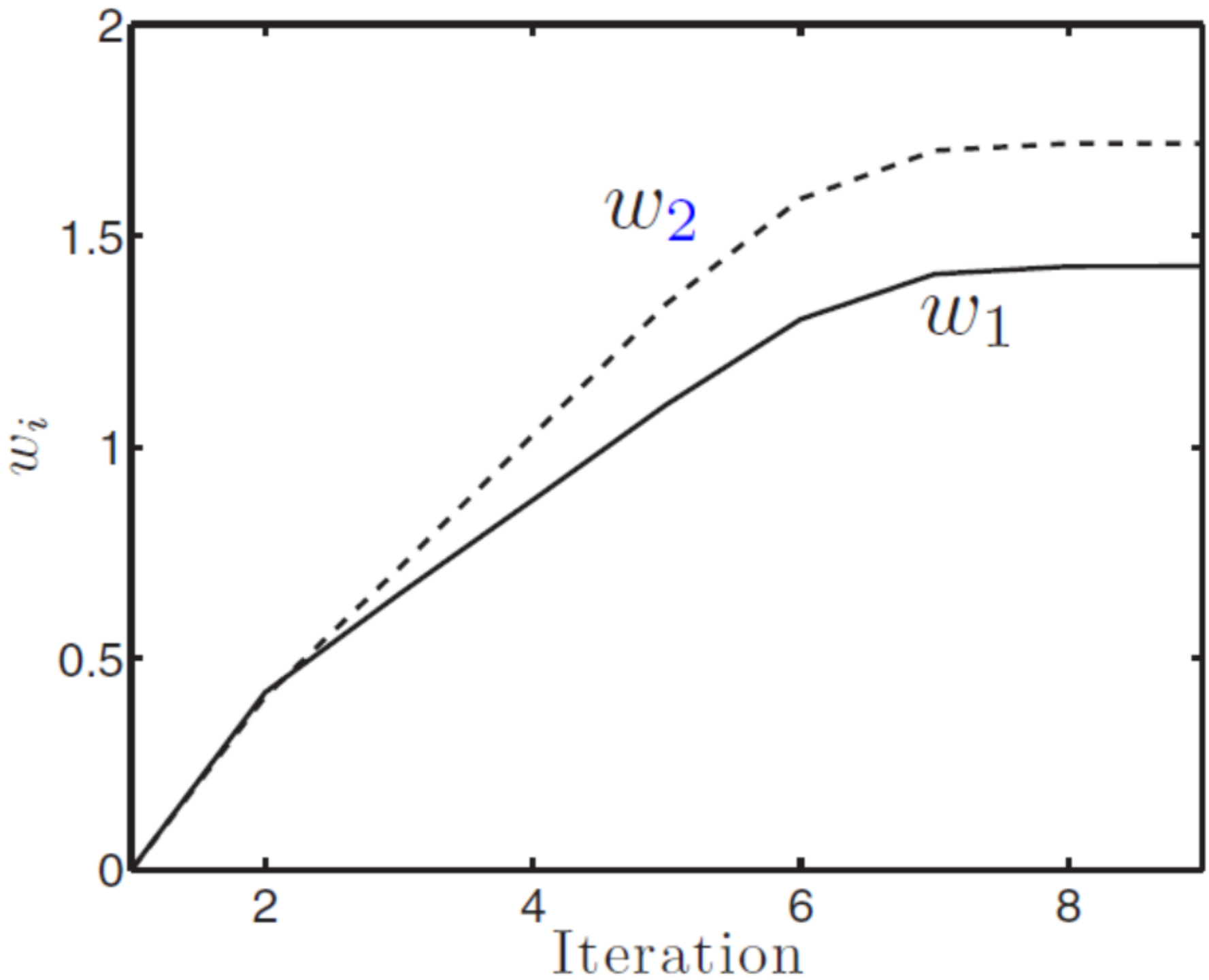
$$\nabla^2 f(X) = H(X) = \begin{bmatrix} \frac{\partial f}{\partial x_0^2} & \frac{\partial f}{\partial x_0 x_1} & \cdots & \frac{\partial f}{\partial x_0 x_n} \\ \frac{\partial f}{\partial x_1 x_0} & \frac{\partial f}{\partial x_2^2} & \cdots & \frac{\partial f}{\partial x_1 x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial x_n x_0} & \frac{\partial f}{\partial x_n x_1} & \cdots & \frac{\partial f}{\partial x_n^2} \end{bmatrix}$$

Convergence

- Because of the uniqueness, whatever value of $\mathbf{w}$ the procedure converges to must correspond to the MAP.

- We perform the Newton-Raphson procedure to find a potential optimal value of $\mathbf{w}$. Starting with $\mathbf{w} = [\mathbf{0}, \mathbf{0}]^{\mathrm{T}}$ and setting $\sigma^2 = 10$, we can converge to the optimal $\mathbf{w}$ after 9 iterations.

- Flaws: 1. Not always global optimum
  2. A point estimate is not so Bayesian.
  3. Decision boundaries are parallel lines.


2
1.5
1
0.5
0
w_i
w_2
w_1
2
4
6
8
Iteration

Decision Boundary

- After having $\widehat{\mathbf{w}}$, we can compute the probability that the response equals 1 for any $\mathbf{x}_{\text{new}}$ (a new set of attributes) as our *probabilistic prediction*:

$$P(T_{\text{new}} = 1|\mathbf{x}_{\text{new}}, \widehat{\mathbf{w}}) = \frac{1}{1 + \exp(-\widehat{\mathbf{w}}^{\mathrm{T}}\mathbf{x}_{\text{new}})}$$

- The set of $\mathbf{x}$ values that correspond to $P(T_{\text{new}} = 1|\mathbf{x}_{\text{new}}, \widehat{\mathbf{w}}) = 0.5$ will form a line that can be thought of as a *decision boundary*: points on one side will belong to one class, and points on the other side to the other class.

$$P(T_{\text{new}} = 1|\mathbf{x}_{\text{new}}, \hat{\mathbf{w}}) = \frac{1}{1 + \exp(-\hat{\mathbf{w}}^{\mathrm{T}}\mathbf{x}_{\text{new}})}$$